

TP2.BDA

1/

```
1 SELECT dept_name
2 FROM department
3 WHERE budget in
4 ( SELECT max ( budget ) FROM department ) ;
```

Results Explain Describe Saved SQL History

DEPT_NAME
Finance

1 rows returned in 0.02 seconds [Download](#)

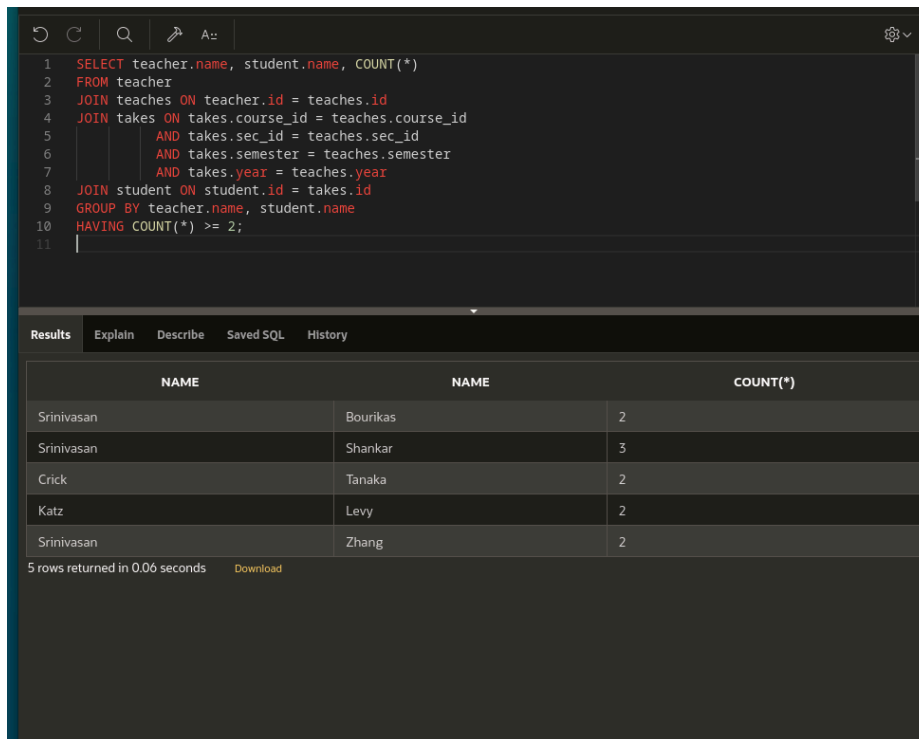
2/

```
1 SELECT A.name, A.salary
2 FROM teacher A
3 WHERE A.salary > (SELECT AVG(B.salary) FROM teacher B);
4 |
```

Results Explain Describe Saved SQL History

NAME	SALARY
Wu	90000
Einstein	95000
Gold	87000
Katz	75000
Singh	80000
Brandt	92000
Kim	80000

7 rows returned in 0.02 seconds [Download](#)



The screenshot shows a SQL IDE interface. The top panel contains a SQL query. The bottom panel shows the results of the query, which is a table with three columns: NAME, NAME, and COUNT(*). The results table has five rows of data. Below the table, it indicates that 5 rows were returned in 0.06 seconds and provides a download link.

```
1 SELECT teacher.name, student.name, COUNT(*)
2 FROM teacher
3 JOIN teaches ON teacher.id = teaches.id
4 JOIN takes ON takes.course_id = teaches.course_id
5             AND takes.sec_id = teaches.sec_id
6             AND takes.semester = teaches.semester
7             AND takes.year = teaches.year
8 JOIN student ON student.id = takes.id
9 GROUP BY teacher.name, student.name
10 HAVING COUNT(*) >= 2;
11
```

NAME	NAME	COUNT(*)
Srinivasan	Bourikas	2
Srinivasan	Shankar	3
Crick	Tanaka	2
Katz	Levy	2
Srinivasan	Zhang	2

5 rows returned in 0.06 seconds [Download](#)

5/

Az

1 SELECT student.id, student.name

2 FROM student

3 MINUS

4 SELECT student.id, student.name

5 FROM student

6 JOIN takes ON takes.id = student.id

7 WHERE takes.year < 2010;

8

Results

Explain

Describe

Saved SQL

History

ID	NAME
19991	Brandt
23121	Chavez
55739	Sanchez
70557	Snow

4 rows returned in 0.01 seconds

Download

6/

Az

1 SELECT * FROM teacher WHERE name LIKE 'E%';

2

Results

Explain

Describe

Saved SQL

History

ID	NAME	DEPT_NAME	SALARY
22222	Einstein	Physics	95000
32543	El Said	History	60000

2 rows returned in 0.00 seconds

Download

```
1 SELECT name
2 FROM teacher T1
3 WHERE 9 = (
4     SELECT COUNT(DISTINCT T2.salary)
5     FROM teacher T2
6     WHERE T2.salary > T1.salary
7 );
8
```

Results Explain Describe Saved SQL History

NAME
Gold

1 rows returned in 0.01 seconds [Download](#)

8/

```
1 SELECT T1.name, T1.salary
2 FROM teacher T1
3 WHERE 2 >= (
4     SELECT COUNT(DISTINCT T2.salary)
5     FROM teacher T2
6     WHERE T2.salary < T1.salary
7 )
8 ORDER BY T1.salary ASC;
9
```

Results Explain Describe Saved SQL History

NAME	SALARY
Mozart	40000
El Said	60000
Califieri	62000

3 rows returned in 0.00 seconds [Download](#)

9/

The screenshot shows a SQL IDE with a query editor and a results pane. The query in the editor is:

```
1 SELECT S.name
2 FROM student S
3 WHERE ('Fall', 2009) IN (
4     SELECT semester, year
5     FROM takes
6     WHERE takes.id = S.id
7 );
8
```

The results pane shows a table with one column, 'NAME', containing 7 rows of student names. Below the table, it states '7 rows returned in 0.02 seconds' and provides a 'Download' link.

NAME
Zhang
Shankar
Peltier
Levy
Williams
Brown
Bourikas

10/

The screenshot shows a SQL IDE with a query editor and a results pane. The query in the editor is:

```
1 SELECT S.name
2 FROM student S
3 WHERE ('Fall', 2009) = SOME (
4     SELECT semester, year
5     FROM takes
6     WHERE takes.id = S.id
7 );
8
```

The results pane shows a table with one column, 'NAME', containing 7 rows of student names. Below the table, it states '7 rows returned in 0.03 seconds' and provides a 'Download' link.

NAME
Zhang
Shankar
Peltier
Levy
Williams
Brown
Bourikas

11/

The screenshot shows a SQL query editor with a dark theme. The query is as follows:

```
1 SELECT DISTINCT student.name
2 FROM takes
3 NATURAL JOIN student
4 WHERE takes.semester = 'Fall' AND takes.year = 2009;
5
```

Below the editor, the 'Results' tab is active, displaying a table with one column named 'NAME'. The table contains seven rows of student names:

NAME
Brown
Zhang
Levy
Bourikas
Shankar
Peltier
Williams

At the bottom of the results section, it states '7 rows returned in 0.00 seconds' and provides a 'Download' link.

12/

```
SELECT name
FROM student S
WHERE EXISTS (
    SELECT 1
    FROM takes T
    WHERE T.id = S.id
    AND T.semester = 'Fall'
    AND T.year = 2009
);
```

13/

```
SELECT A.name, B.name
FROM (SELECT * FROM student NATURAL JOIN takes) A,
     (SELECT * FROM student NATURAL JOIN takes) B
WHERE A.course_id =
      B.course_id AND A.sec_id =
      B.sec_id AND A.semester =
      B.semester AND A.year = B.year
      AND A.id <> B.id
      AND A.name < B.name
GROUP BY A.name, B.name
HAVING COUNT(*) >= 1;
```

14/

```
SELECT teacher.name, COUNT(*) AS
total_students FROM takes
INNER JOIN teaches USING (course_id, sec_id, semester, year)
INNER JOIN teacher ON teaches.id = teacher.id
GROUP BY teacher.name, teacher.id
ORDER BY COUNT(*) DESC;
```

15/

```
SELECT teacher.name, COUNT(*) AS
total_students FROM takes
INNER JOIN teaches USING (course_id, sec_id, semester, year)
INNER JOIN teacher ON teaches.id = teacher.id
GROUP BY teacher.name, teacher.id
ORDER BY COUNT(*) DESC;
```

16/

```
WITH mytakes (id, course_id, sec_id, semester, year, grade) AS (
    SELECT id, course_id, sec_id, semester, year, grade
    FROM takes
    WHERE grade = 'A'
)
SELECT teacher.name, COUNT(course_id) AS total_A
FROM (mytakes INNER JOIN teaches USING (course_id, sec_id, semester, year))
RIGHT OUTER JOIN teacher ON teaches.id = teacher.id
GROUP BY teacher.name, teacher.id
ORDER BY COUNT(course_id) DESC;
```

17/

```
SELECT teacher.name, student.name, COUNT(*)
FROM (teacher NATURAL JOIN teaches)
INNER JOIN (takes NATURAL JOIN student)
USING (course_id, sec_id, semester, year)
GROUP BY teacher.name, student.name;
```

18/

```
SELECT teacher.name AS tn, student.name AS sn
FROM (teacher NATURAL JOIN teaches)
INNER JOIN (takes NATURAL JOIN student)
USING (course_id, sec_id, semester, year)
GROUP BY teacher.name, student.name
HAVING COUNT(*) >= 2;
```

Exo: 02

1. Schéma $R(A,B,C)$
 $F = \{A \rightarrow B; B \rightarrow C\}$

- Calcul de A^+
À partir de AA :
 - $A \rightarrow B$ donne BB
 - Puis, $B \rightarrow C$ donne CCCAinsi, $A^+ = \{A, B, C\}$
 $A^+ = \{A, B, C\}$.
La seule clé minimale est donc $\{A\}$.
- FD $A \rightarrow B$:
LHS A est une clé $\rightarrow$ Pas de problème.
- FD $B \rightarrow C$:
Ici, $B^+ = \{B, C\}$ et BB n'est pas une clé (ne contient pas AA).
 $\rightarrow$ pas de BCNF.

1. $R_1(B,C)$ avec $B \rightarrow C$
 - Dans R_1 , la clé est B (car $B^+ = \{B, C\}$).
2. $R_2(A,B)$
 - Dans R_2 , la FD $A \rightarrow B$ s'y applique et la clé est A (puisque $A^+ = \{A, B\}$).

Cette décomposition est sans perte et chacune des relations est en BCNF.

2. Schéma $R(A,B,C)$ avec $F = \{A \rightarrow C; A \rightarrow B\}$

Identification de la clé

- Calcul de A^+
À partir de AA :
 - $A \rightarrow C$ donne CC

- $A \rightarrow B \setminus \text{to } B$ donne BB
Ainsi, $A^+ = \{A, B, C\}$ $A^+ = \{A, B, C\}$.
La clé minimale est donc $\{A\} \setminus \{A\}$.

Vérification de la BCNF

- Les deux FD ont pour LHS la clé AA .
→ La relation est déjà en **BCNF** (et donc en forme normale avancée).

Aucune décomposition n'est nécessaire.

3.

Identification des clés

- La FD $A, B \rightarrow C \setminus \text{to } C$ montre que $\{A, B\} \setminus \{A, B\}$ détermine CC et donc $\{A, B\} \setminus \{A, B\}$ est une clé.
- En outre, grâce à $C \rightarrow B \setminus \text{to } B$ on peut voir que $\{A, C\} \setminus \{A, C\}$ est aussi une clé car :
 - À partir de $\{A, C\} \setminus \{A, C\}$, on déduit BB (par $C \rightarrow B \setminus \text{to } B$) et ainsi $A, B \rightarrow C \setminus \text{to } C$ redonne CC .
Ainsi, les clés minimales sont $\{A, B\} \setminus \{A, B\}$ et $\{A, C\} \setminus \{A, C\}$.

Vérification des formes normales

- FD $A, B \rightarrow C \setminus \text{to } C$:
Le LHS $\{A, B\} \setminus \{A, B\}$ est une clé → Pas de problème.
- FD $C \rightarrow B \setminus \text{to } B$:
Ici, CC seul n'est pas une clé (puisque $C + C^+$ donne CC et BB via $C \rightarrow B \setminus \text{to } B$, mais il manque AA).
→ Violation de la BCNF.

Décomposition en BCNF

1. $R_1(C, B) \setminus R_1(C, B)$
 - La FD $C \rightarrow B \setminus \text{to } B$ s'y applique et la clé de $R_1 \setminus R_1$ est CC (car $C^+ = \{C, B\}$ $C^+ = \{C, B\}$).
2. $R_2(A, C) \setminus R_2(A, C)$
 - La décomposition conserve l'attribut CC pour joindre $R_1 \setminus R_1$ et $R_2 \setminus R_2$.

- Dans $R2R_2$, aucune FD non triviale issue de FF n'est présente, donc la clé est $\{A, C\} \setminus \{A, C\}$ (ou l'ensemble complet).

Vérification de la décomposition :

- L'attribut commun est CC, qui est une clé dans $R1R_1$.

Exo:2

(a) Fermeture de B et de $\{A, B\}$

Fermeture de l'attribut B (notée B^+)

1. Initialisation :
 $B^+ = \{B\}$.
2. Application de $B \rightarrow D$:
Ajoute $D \rightarrow B^+ = \{B, D\}$.
3. Application de $D \rightarrow A$:
 D dans B^+ permet d'ajouter $A \rightarrow B^+ = \{A, B, D\}$.
4. Application de $A \rightarrow B, C, D$:
Depuis A , on ajoute C (B et D déjà présents) $\rightarrow B^+ = \{A, B, C, D\}$.
5. Application de $B, C \rightarrow D, E$:
Comme B^+ contient B et C , on ajoute $E \rightarrow B^+ = \{A, B, C, D, E\}$.

(b)

Pour montrer que $\{A, F\}$ est une super-clé, il faut prouver que sa fermeture couvre tous les attributs de R.

Calcul de $(A, F)^+$

1. **Initialisation :**

$$(A, F)^+ = \{A, F\}.$$

2. **Application de $A \rightarrow B, C, D$:**

Ajoute B, C et D $\rightarrow (A, F)^+ = \{A, B, C, D, F\}.$

3. **Application de $D \rightarrow A$:**

$D \rightarrow A$, mais A est déjà présent.

4. **Application de $B, C \rightarrow D, E$:**

Puisque B et C sont dans $(A, F)^+$, on ajoute E $\rightarrow (A, F)^+ = \{A, B, C, D, E, F\}.$

On obtient ainsi

$$(A, F)^+ = \{A, B, C, D, E, F\} = R.$$

Donc, $\{A, F\}$ est une super-clé de R.

(c)

1. FD1 : $A \rightarrow B, C, D$

- $A^+ = \{A, B, C, D, E\}$ (F n'étant pas obtenu).
- A n'est donc pas une super-clé de R (car F est manquant).
 $\rightarrow$ Violation de la BCNF.

2. FD2 : $B, C \rightarrow D, E$

- $(B, C)^+ = \{A, B, C, D, E\}$ (encore F absent).
 $\rightarrow$ Violation de la BCNF.

3. FD3 : $B \rightarrow D$

- $B^+ = \{A, B, C, D, E\}$ (F manquant).
 $\rightarrow$ Violation de la BCNF.

4. FD4 : $D \rightarrow A$

- $D^+ = \{A, B, C, D, E\}$ (F manquant).

: $A \rightarrow B, C, D$ (avec A n'étant pas une clé de R).

- Créer R_1 avec les attributs impliqués dans la FD :
 $R_1(A, B, C, D)$.
- Créer R_2 qui contient l'attribut A (pour assurer la perte minimale) et les attributs restants de R.
R a $\{A, B, C, D, E, F\}$, et R_1 contient $\{A, B, C, D\}$ donc
 $R_2(A, E, F)$.
- Pour $R_1(A, B, C, D)$:
Les dépendances applicables sont :
 - $A \rightarrow B, C, D$
 - $D \rightarrow A$ (et éventuellement $B \rightarrow D$ si l'on restreint FD3 à R_1) On vérifie :
 - A^+ dans $R_1 = \{A, B, C, D\} \rightarrow A$ est une clé de R_1 .
 - De même, d'autres dépendances ($B \rightarrow D, D \rightarrow A$) se retrouvent avec des LHS qui sont des clés (on peut démontrer que B et D sont aussi candidats).
 $\rightarrow R_1$ est en BCNF.
- Pour $R_2(A, E, F)$:
Dans R_2 , aucune FD non triviale de F ne s'applique (les FDs de R impliquant B, C ou D ne concernent pas R_2).
 $\rightarrow R_2$ est en BCNF.
- Perte minimale (lossless join) :
La relation R_1 et R_2 se rejoignent sur l'attribut A qui est clé de R_1 .
 $\rightarrow$ La décomposition est sans perte.

- Préservation des dépendances :
Certaines dépendances (par exemple, $B, C \rightarrow D, E$) ne sont pas entièrement préservées dans l'une ou l'autre des relations. Or, en BCNF, la préservation des dépendances n'est pas toujours possible.

3(a) Décomposition de $R(A, B, C, D, E)$ en $R_1(A, B, C)$ et $R_2(A, D, E)$

- Intersection :
Les deux relations partagent l'attribut A ($R_1 \cap R_2 = \{A\}$).
- Critère de perte (théorème de la décomposition sans perte) :
La décomposition est sans perte si l'intersection (ici $\{A\}$) est une clé pour l'un des schémas décomposés.
Par exemple, si dans la relation R (ou dans l'une des projections) l'attribut A permet d'identifier de manière unique le tuple, alors pour chaque valeur de A, il existe un seul tuple dans R_1 et un seul dans R_2 .
Le joint naturel sur A réunira alors exactement les informations d'origine, c'est-à-dire :
 - Pour tout tuple r de R , ses projections $r_1 = \pi_{(A,B,C)}(r)$ et $r_2 = \pi_{(A,D,E)}(r)$ vérifient $r = r_1 \bowtie r_2$.

- Conclusion :
On peut conclure que la décomposition est sans perte (lossless) si A est une clé dans au moins l'un des schémas (ce qui est souvent le cas dans ce type de décomposition).

3(b) Décomposition de R(A, B, C, D, E) en R1(A, B, C) et R2(C, D, E)

- Intersection :
Ici, l'intersection est {C} ($R1 \cap R2 = \{C\}$).
- Analyse du critère :
Pour que la décomposition soit sans perte, il faudrait que {C} soit une clé (ou super-clé) dans l'un des schémas R1 ou R2.
Or, en général, l'attribut C seul ne suffit pas à identifier de manière unique un tuple dans R1 ni dans R2.

- Exemple
Considérons l'instance suivante de R :

A	B	C	D	E
1	2	3	4	5
6	7	3	8	9

○

Projection sur R1 (A, B, C) :

A	B	C
1	2	3
6	7	3

○

Projection sur R2 (C, D, E) :

C	D	E
3	4	5
3	8	9

- Lors du joint naturel sur C, on obtient toutes les combinaisons possibles entre les tuples de R1 et R2 ayant C = 3 :

A	B	C	D	E
---	---	---	---	---

1 2 3 4 5

1 2 3 8 9

6 7 3 4 5

6 7 3 8 9

- Les tuples « spuriques » (qui n'étaient pas dans la relation R d'origine) montrent que l'information n'est pas préservée.
- Conclusion :
La décomposition en $R_1(A, B, C)$ et $R_2(C, D, E)$ est avec perte car l'intersection $\{C\}$ ne constitue pas une clé et le joint naturel sur C produit des tuples supplémentaires non présents dans la relation originale.

EXO:4

```
def printDependencies(F: "list of dependencies"): for  
    alpha, beta in F:  
        print("\t", alpha, " --> ", beta)
```


2.

```
import itertools
```

```
def powerSet(inputset: set):
```

```
    _result = []
```

```
    for r in range(1, len(inputset) + 1):
```

```
        # itertools.combinations retourne des tuples que nous convertissons en set
```

```
        _result += list(map(set, itertools.combinations(inputset, r)))
```

```
    return _result
```

```
# Exemple d'utilisation :
```

```
myset = {'A', 'B', 'C'}
```

```
all_subsets = powerSet(myset)
```

```
print("Tous les sous-ensembles non vides de", myset, ":", all_subsets)
```

Ces deux procédures répondent aux exigences de l'exercice.

```
import itertools
```

```
# Question 4 : Fermeture d'un ensemble d'attributs K sous F def
```

```
closure(F, K):
```

```
    """
```

Retourne la fermeture de l'ensemble d'attributs K sous l'ensemble de dépendances fonctionnelles F.

Paramètres :

- F : liste de dépendances, chaque dépendance est représentée par un tuple (X, Y) où X et Y sont des ensembles.
- K : ensemble d'attributs (set)

Retour :

- closure_set : ensemble des attributs déduits de K (K^+)

```
    """
```

```
closure_set = set(K) # On part de K
```

```
changed = True
```

```
while changed:
```

```
    changed = False
```

```
    for (X, Y) in F:
```

```
        # Si le côté gauche X est inclus dans la fermeture, alors on peut ajouter Y. if
```

```
        X.issubset(closure_set) and not Y.issubset(closure_set):
```

```
            closure_set |= Y
```

```
            changed = True
```

```
    return closure_set
```

```
# Fonction utilitaire : calcul du power set (ensemble des sous-ensembles) def
```

```
powerSet(inputset: set):
```

```
    """
```

Retourne le power set (l'ensemble de tous les sous-ensembles) de inputset, y compris l'ensemble vide.

Paramètre :

- inputset : ensemble (set)

Retour :

- liste de sous-ensembles (chacun est un set)

"""

result = []

for r in range(0, len(inputset) + 1):

 result += list(map(set, itertools.combinations(inputset, r)))

return result

Question 5 : Clôture de l'ensemble de dépendances fonctionnelles F (F^+) def

fdClosure(F):

"""

Retourne la clôture F^+ de l'ensemble de dépendances fonctionnelles F.

F^+ est l'ensemble de toutes les dépendances fonctionnelles (non triviales) qui peuvent être déduites de F.

Méthode : on considère l'univers des attributs U (issus de F), puis

pour chaque sous-ensemble X de U, on calcule X^+ et on en déduit les dépendances

$X \rightarrow Y$

pour tout $Y \subseteq X^+$ avec Y n'étant pas contenu dans X (pour obtenir les dépendances non triviales).

Paramètre :

- F : liste de dépendances (chaque dépendance est un tuple (X, Y))

Retour :

- Un ensemble de dépendances sous forme de tuples (frozenset(X), frozenset(Y))

"""

Déterminer l'univers U des attributs présents dans F. U =

set()

for (X, Y) in F:

 U |= X

 U |= Y

result = set()

all_subsets = powerSet(U) for

X in all_subsets:

 X_closure = closure(F, X)

 # Pour chaque sous-ensemble Y de X_{closure} tel que Y n'est pas inclus dans X (pour éviter les dépendances triviales)

 for Y in powerSet(X_closure):

 if not Y.issubset(X): # On ne retient que les dépendances non triviales $X \rightarrow Y$

 result.add((frozenset(X), frozenset(Y)))

return result

Question 6 : Déterminer si α détermine fonctionnellement β sous F def
determines(F, alpha: set, beta: set):

"""

Retourne True si l'ensemble d'attributs alpha détermine fonctionnellement beta sous F,
c'est-à-dire si $\beta \subseteq (\alpha)^+$.

Paramètres :

- F : liste de dépendances fonctionnelles
- alpha : ensemble d'attributs (set) servant de déterminant
- beta : ensemble d'attributs (set) dont on vérifie la dépendance

Retour :

- True si $\beta \subseteq \text{closure}(F, \alpha)$, sinon False.

"""

return beta.issubset(closure(F, alpha))

Question 7 : Déterminer si K est une super-clé pour la relation R def
isSuperKey(F, R: set, K: set):

"""

Retourne True si K est une super-clé pour la relation R (ensemble des attributs de
R)

sous l'ensemble de dépendances fonctionnelles F. Cela revient à vérifier que la
fermeture de K est égale à R.

Paramètres :

- F : liste de dépendances fonctionnelles
- R : ensemble des attributs de la relation (set)
- K : ensemble d'attributs candidat (set)

Retour :

- True si $\text{closure}(F, K) == R$, sinon False.

"""

return closure(F, K) == R

Question 8 : Déterminer si K est une clé candidate pour la relation R def
isCandidateKey(F, R: set, K: set):

"""

Retourne True si K est une clé candidate pour la relation R sous l'ensemble de
dépendances F.

K est une clé candidate si :

1. K est une super-clé ($\text{closure}(F, K) == R$)
2. K est minimale, c'est-à-dire qu'aucun sous-ensemble propre de K n'est une super-clé.

Paramètres :

- F : liste de dépendances fonctionnelles
- R : ensemble des attributs de la relation (set)
- K : ensemble d'attributs candidat (set)

Retour :

- True si K est une clé candidate, sinon False.

"""

not isSuperKey(F, R, K):

 return False

super-clé.

for a in K:

 if isSuperKey(F, R, K - {a}):

 return False

return True

def allCandidateKeys(F, R: set): """

 Retourne la liste de toutes les clés candidates de la relation R sous F.

Paramètres:

- F : liste de dépendances fonctionnelles, où chaque dépendance est un tuple (X, Y)

 avec X et Y des ensembles.

- R : ensemble des attributs de la relation.

Retour:

- Liste (de sets) de clés candidates.

"""

candidate_keys = []

```

for subset in powerSet(R):
    if subset and isCandidateKey(F, R, subset):
        candidate_keys.append(subset)
return candidate_keys

```

```

def allSuperKeys(F, R: set):
    """

```

Retourne la liste de toutes les super-clés de la relation R sous F.

Paramètres:

- F : liste de dépendances fonctionnelles (chaque FD est un tuple (X, Y)).
- R : ensemble des attributs de la relation.

Retour:

- Liste (de sets) de super-clés. """

```

super_keys = []
for subset in powerSet(R):
    if subset and isSuperKey(F, R, subset):
        super_keys.append(subset)
return super_keys

```

```

def findCandidateKey(F, R: set): """

```

Retourne une clé candidate de la relation R sous F.

L'algorithme itère sur les sous-ensembles de R triés par taille (les plus petits d'abord) et retourne le premier qui est une clé candidate.

Paramètres:

- F : liste de dépendances fonctionnelles (chaque FD est un tuple (X, Y)).
- R : ensemble des attributs de la relation.

Retour:

- Une clé candidate (set) si elle existe, sinon None.

```

"""

```

Génère tous les sous-ensembles non vides de R.

```

subsets = [s for s in powerSet(R) if s]

```

Trie les sous-ensembles par taille croissante pour trouver la clé minimale. subsets =

```

sorted(subsets, key=lambda s: len(s))

```

```

for s in subsets:

```

```

    if isCandidateKey(F, R, s):

```

```

        return s

```

```

return None

```

```

# -----
# Exemples d'utilisation
if __name__ == "__main__":
    # On reprend l'exemple de dépendances fonctionnelles utilisé précédemment.
    mydependencies = [
        ( {'A'}, {'B'} ),    # A -> B
        ( {'A'}, {'C'} ),    # A -> C
        ( {'C', 'G'}, {'H'} ),# CG -> H
        ( {'C', 'G'}, {'I'} ),# CG -> I
        ( {'B'}, {'H'} )     # B -> H
    ]

    # L'univers des attributs de la relation R. R
    = {'A', 'B', 'C', 'G', 'H', 'I'}

    # 9. Liste de toutes les clés candidates
    candidate_keys = allCandidateKeys(mydependencies, R) print("Clés
    candidates de R :", candidate_keys)

    # 10. Liste de toutes les super-clés
    super_keys = allSuperKeys(mydependencies, R)
    print("Super-clés de R :", super_keys)

    # 11. Une clé candidate (la première trouvée) candidate =
    findCandidateKey(mydependencies, R) print("Une clé
    candidate trouvée pour R :", candidate)

```

12. Vérifier si une relation R est en BCNF

Une relation R est en BCNF si, pour toute dépendance fonctionnelle non triviale $X \rightarrow Y$ telle que $Y \not\subseteq X$ (c'est-à-dire telle que $Y \subseteq X \cup Y$ et $Y \not\subseteq X$) dont les attributs sont inclus dans R, X est une super-clé de R. Nous définissons :

```
def isBCNF(R: set, F: list) -> bool: """
```

```
    Retourne True si la relation R est en BCNF par rapport à l'ensemble de
    dépendances F.
```

```
    -
```

```
    """
```

```
    for (X, Y) in F:
```

```
        # Considérer seulement les dépendances dont  $X \cup Y \subseteq R$  if
```

```
        X.union(Y).issubset(R):
```

```
            # Si la dépendance est non triviale if
```

```
            not Y.issubset(X):
```

```
                # Vérifier que X est une super-clé de R
```

```
                if not isSuperKey(F, R, X):
```

```
                    return False
```

```
    return True
```

13.

Ici, le schéma est représenté par une liste de relations (chaque relation étant un ensemble d'attributs). Pour chaque relation R de T, on considère les dépendances locales $F_{local} = \{ (X, Y) \in F \mid X \cup Y \subseteq R \}$ et on vérifie si R est en BCNF.


```
def schemalsBCNF(T: list, F: list) -> bool: """
    Retourne True si chacune des relations de la liste T est en BCNF par
    rapport à F.

    Paramètres:
    - T : liste de relations, chaque relation est un set d'attributs.
    - F : liste de dépendances fonctionnelles (chacune (X, Y) avec X, Y sets).
    """
    for R in T:
        # Extraire les dépendances pertinentes pour R
        F_local = [(X, Y) for (X, Y) in F if X.union(Y).issubset(R)]
        if not isBCNF(R, F_local):
            return False
    return True
```

14.

- $R_1 = X \cup Y$
- $R_2 = R - (Y - X)$ (c'est-à-dire, $R_2 = (R \setminus (Y \setminus X))$), ce qui revient à conserver X dans R_2)

```
def bcnfDecomposition(F: list, T: list) -> list: """
    Implémente l'algorithme de décomposition en BCNF.
```

Paramètres:

- F : liste de dépendances fonctionnelles, chacune (X, Y) avec X, Y sets.
- T : liste initiale de relations (chaque relation est un set d'attributs).

Retour:

- Une nouvelle liste de relations, toutes en BCNF.

```
"""
result = T.copy()
changed = True
while changed:
    changed = False
    new_result = []
    for R in result:
```

```

# Extraire les dépendances pertinentes pour R
F_local = [(X, Y) for (X, Y) in F if X.union(Y).issubset(R)] if
isBCNF(R, F_local):
    new_result.append(R)
else:
    # Chercher une dépendance violant la BCNF dans F_local viol =
    None
    for (X, Y) in F_local:
        if not Y.issubset(X): # non triviale if
            not isSuperKey(F_local, R, X):
                viol = (X, Y)
                break
    if viol is None:
        new_result.append(R)
    else:
        X, Y = viol
        # Décomposition :
        R1 = X.union(Y)
        R2 = R - (Y - X) # équivalent à  $(R \setminus (Y \setminus X))$ 
        new_result.extend([R1, R2])
        changed = True
    result = new_result
return result

```